

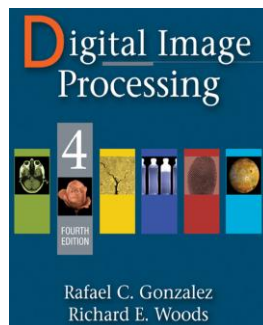


دانشگاه اصفهان

دانشکده مهندسی کامپیوتر

گزارش تمرین دوم – تصویرپردازی رقمی

مورفولوژی



پدیدآورنده:

محمد امین کیانی

۴۰۴۳۶۴۴۰۰۸

دانشجوی کارشناسی ارشد، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،

Aminkianiworkeng@gmail.com

استاد درس: دکتر نقش نیلچی

نیمسال دوم تحصیلی ۱۴۰۴-۰۵

فهرست مطالب

مستندات	۳
۰- مسئله و تحلیل کلی آن:	۳
۱- تمرین اول: Dilation	۶
۲- تمرین دوم: Erosion	۱۰
۳- تمرین سوم: Opening	۱۳
۴- تمرین چهارم: Closing	۱۶
۵- تمرین پنجم: Hit-or-Miss Transform	۱۹
۶- تمرین ششم: باقی عملیات‌های ریخت‌شناسی	۲۲
۷- تمرین هفتم: انجام تمامی عملیات‌ها بر روی هر تصویر دلخواه	۴۱
۸- مراجع	۴۵

مستندات

۰- مسئله و تحلیل کلی آن:

در این مجموعه تمرین، هدف این است که عملیات پایه‌ی مورفولوژی ریاضی را روی تصاویر دودویی و خاکستری ببینیم و بفهمیم هر عمل چه اثری روی ساختارهای روشن/تیره‌ی تصویر می‌گذارد. برای هر مثال، ترجمه‌ی مستقیم منطق MATLAB به Python انجام شده است تا همان رفتار مفهومی حفظ شود، اما کد برای اجرای راحت در Google Colab آماده شده است. ساختارهای مورفولوژیک اینجا فقط ابزار نمایش نیستند؛ آن‌ها برای اتصال قطعات شکسته، حذف اجسام کوچک، صاف کردن مرزها، و تشخیص الگوهای خاص استفاده می‌شوند.

منابع تصویری مورد استفاده

تصویر circles از یک مخزن عمومی GitHub گرفته شده است.

تصویر blobs از مخزن عمومی samples.fiji.sc دریافت شده است.

تصویر snowflakes از یک مخزن عمومی GitHub گرفته شده است.

تصویر cameraman از یک مجموعه‌ی استاندارد عمومی تصویر برای پردازش تصویر دریافت شده است.

در نوت‌بوک، برای reproducibility، همه‌ی این تصاویر از آدرس‌های عمومی قابل دانلود شده‌اند و نیازی به آپلود دستی فایل‌ها نیست. همچنین در Colab کافی است این بسته‌ها نصب شوند:

```
!pip -q install numpy matplotlib pillow scipy scikit-image ...
```

MATLAB	Python
zeros(256,256)	<code>np.zeros((256, 256), dtype=bool)</code>
B(i,j)=1	<code>B[i, j] = True</code>
ones(k)	<code>np.ones((k, k), dtype=bool)</code>
imdilate	<code>scipy.ndimage.binary_dilation</code>
imerode	<code>scipy.ndimage.binary_erosion</code>
im2bw(I,0.3)	<code>I > int(0.3*255)</code>
imshow(...), title(...)	<code>matplotlib.pyplot.imshow(..., cmap='gray')</code>

Dilation: $A \oplus B$

Erosion: $A \ominus B$

Closing: $A \bullet B = (A \oplus B) \ominus B$

Opening: $A \circ B = (A \ominus B) \oplus B$

Hit-and-Miss: $A \otimes (B1, B2) = (A \ominus B1) \cap (Ac \ominus B2)$

- ایده‌ی اصلی پردازش مورفولوژیک این است که به‌جای کار با شدت روشنایی پیکسل‌ها، با شکل و ساختار اشیا کار کنیم.

- در تصاویر باینری، هر پیکسل فقط دو حالت دارد:

• ۱ یا سفید: عضو شیء

• ۰ یا سیاه: پس‌زمینه

- عملیات مورفولوژیک با استفاده از یک عنصر کوچک به نام **Structuring Element** یا عنصر سازنده انجام می‌شوند. این عنصر مثل یک «الگو» یا «قالب» است که روی تصویر حرکت می‌کند و مشخص می‌کند چه اتفاقی روی شکل‌ها بیفتد. که در این تمرین:

`strel('disk', r)` در MATLAB یعنی یک عنصر سازنده‌ی دیسکی (دایره‌ای) با شعاع `r`.

چرا دیسک؟

چون وقتی شکل‌ها یا اشیای تصویر حالت گرد، تقریباً گرد، یا بدون جهت خاص دارند، عنصر دایره‌ای از نظر هندسی مناسب‌تر است. برخلاف SE مربعی، دیسک در همه‌ی جهت‌ها اثر تقریباً یکسان دارد و جهت‌دار نیست.

چرا عدد ۵ یا ۱۰؟

این عدد شعاع است، یعنی اندازه‌ی عنصر سازنده را تعیین می‌کند.

• شعاع کوچک‌تر `<==` اثر ملایم‌تر

• شعاع بزرگ‌تر `<==` اثر شدیدتر

مثال:

• در **Closing** روی `circles.png`، از شعاع ۱۰ استفاده می‌کنیم چون هدف این است که شکاف‌های بزرگ‌تر بین بخش‌های دایره‌ها پر شود و لبه‌ها هم صاف‌تر شوند.

• در **Opening** روی `blobs.png` و `snowflakes.png`، از شعاع ۵ استفاده می‌کنیم چون می‌خواهیم نقاط و اجزای کوچک‌تر از این اندازه حذف شوند ولی ساختار اصلی حفظ شود. پس عدد شعاع را شانس انتخاب نمی‌کنیم؛ این عدد به مقیاس نویز یا شکاف‌ها در تصویر بستگی دارد.

۱- تمرین اول: Dilation

هدف

گسترش نواحی سفید، پرکردن گپ‌ها و بزرگ‌تر کردن اجسام.

ایده‌ی ریاضی

اگر A تصویر باینری و B عنصر سازنده باشد، آنگاه: $A \oplus B$

دیلایشن یعنی اگر عنصر سازنده با بخشی از شیء برخورد کند، آن قسمت شیء را گسترش می‌دهد.

کد

۱) ساخت تصویر اولیه

در کد ابتدا یک تصویر 256×256 می‌سازیم که $B = \text{zeros}(256, 256)$ یعنی کل تصویر ابتدا سیاه است.

۲) رسم یک مربع سفید

```
for(i=۱۲۸:۱۶۰)
    for(j=۱۲۸:۱۶۰)
        B(i,j)=۱;
    end
end
```

این حلقه یک مربع سفید در وسط تصویر ایجاد می‌کند زیرا می‌خواهیم اثر dilation را روی یک شیء ساده و قابل مشاهده ببینیم.

۳) رسم یک ناحیه‌ی سفید دیگر

```
for(i=۲۰:۲۵)
    for(j=۳۰:۱۹۰)
        B(i,j)=۱;
    end
end
```

این بخش یک نوار سفید باریک اضافه می‌کند تا ببینیم dilation فقط روی یک شکل ساده عمل نمی‌کند، بلکه روی اجسام با فرم متفاوت هم اثر دارد.

۴) اضافه کردن پیکسل‌های تصادفی

```
While(i<۴۰)
    x=uint8(rand*۲۵۵);
    y=uint8(rand*۲۵۵);
    B(x,y)=۱;
    i=i+۱;
end
```

این قسمت ۴۰ پیکسل تصادفی سفید اضافه می‌کند زیرا می‌خواهیم نشان دهیم dilation چگونه **نویزهای سفید** یا نقاط پراکنده را هم بزرگ می‌کند.

در Python/Colab این بخش با random seed قابل تکرار می‌شود، اما چون تصادفی است، اجرای چندباره نتیجه‌های متفاوت می‌دهد؛ به همین دلیل باید حداقل ۳ بار اجرا شود(مطابق خواسته‌ی مسئله).

(۵) چرا kernel های ۳، ۵، ۷، ۹؟

$K_3 = \text{ones}(3);$

$K_5 = \text{ones}(5);$

$K_7 = \text{ones}(7);$

$K_9 = \text{ones}(9);$

این‌ها چهار structuring element مربعی هستند و تمامی عناصرشان ۱ است، یعنی کل ناحیه فعال است.

چرا از **ones** استفاده شده است؟

چون در این مثال هدف فقط دیدن اثر اندازه‌ی **SE** است، نه شکل خاص آن. **SE** مربعی و پر، ساده‌ترین انتخاب برای مشاهده‌ی تفاوت ابعاد است.

نتیجه‌ی این تغییر اندازه چیست؟

• $3 \times 3 \rightarrow$ اثر کم

• $5 \times 5 \rightarrow$ اثر بیشتر

• $7 \times 7 \rightarrow$ بزرگ‌سازی شدیدتر

• $9 \times 9 \rightarrow$ بیشترین گسترش

هرچه **SE** بزرگ‌تر باشد، **dilation** قوی‌تر است.

(۶) **dilation** روی تصویر

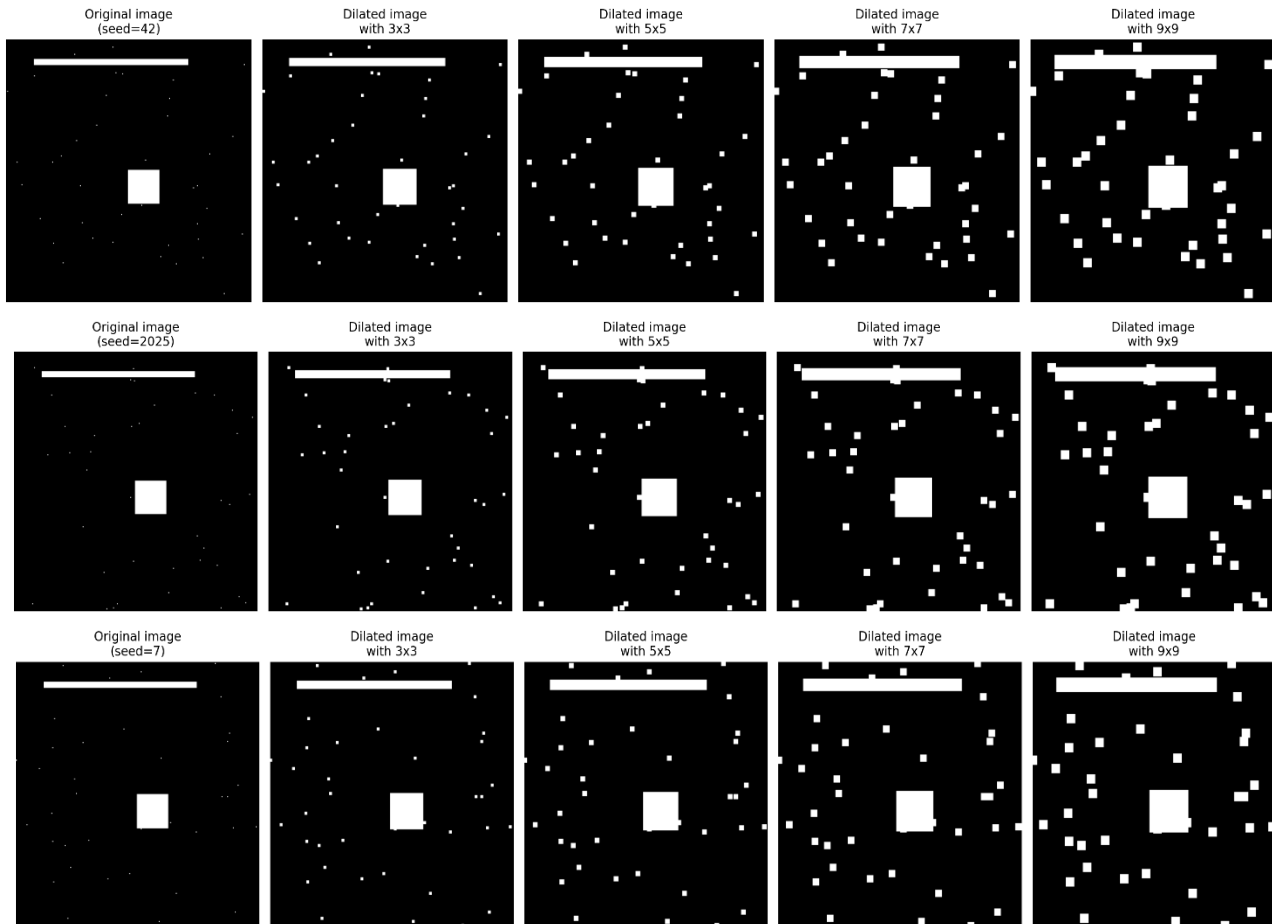
$B_3 = \text{imdilate}(B, K_3);$

$B_5 = \text{imdilate}(B, K_5);$

$B_7 = \text{imdilate}(B, K_7);$

$B_9 = \text{imdilate}(B, K_9);$

در اینجا همان تصویر اولیه با چهار SE مختلف dilate می‌شود.



معنای عملی:

- اجسام سفید بزرگ‌تر می‌شوند.
- فاصله‌ی بین اجسام کوچک می‌شود.
- نقاط پراکنده هم بزرگ‌تر و واضح‌تر می‌شوند.
- لبه‌ها ضخیم‌تر می‌شوند.

کاربرد dilation چیست؟

- پر کردن فاصله‌های کوچک
- وصل کردن اجسام نزدیک به هم
- تقویت نواحی روشن در تصویر باینری
- پیش‌پردازش برای تشخیص شکل‌ها

۲- تمرین دوم: Erosion

هدف

کوچک کردن اجسام سفید، حذف اجسام کوچک و جدا کردن اتصال‌های باریک.

ایده‌ی ریاضی

$A \ominus B$ یعنی فقط آن قسمت‌هایی از شیء باقی می‌مانند که SE به‌طور کامل داخل آن‌ها جا شود.

کد

(۱) خواندن تصویر

```
I = imread('cameraman.tif');
```

در نسخه‌ی پایتون، این تصویر از منبع عمومی بارگذاری می‌شود.

چرا **cameraman**؟ چون یکی از تصویرهای کلاسیک در پردازش تصویر است و برای نمایش مفاهیم به‌خوبی شناخته شده‌است.

۲) تبدیل به باینری

$B = \text{im2bw}(I, 0.3);$

این خط شدت خاکستری را با آستانه‌ی ۰.۳ به باینری تبدیل می‌کند.

چرا این کار لازم است؟ چون عملیات مورفولوژیک کلاسیک روی تصویر باینری تعریف می‌شوند و ما می‌خواهیم تصویر به دو بخش foreground/background تبدیل شود تا erosion روی ساختارها اعمال شود.

۳) ایجاد SE های مختلف

$K_3 = \text{ones}(3);$

$K_5 = \text{ones}(5);$

$K_7 = \text{ones}(7);$

$K_9 = \text{ones}(9);$

این بار هم اندازه‌ی SE مهم است، چون erosion با SE بزرگ‌تر، بخش بیشتری از شیء را حذف می‌کند.

۴) اعمال erosion

$B_3 = \text{imerode}(B, K_3);$

$B_5 = \text{imerode}(B, K_5);$

$B_7 = \text{imerode}(B, K_7);$

$B_9 = \text{imerode}(B, K_9);$

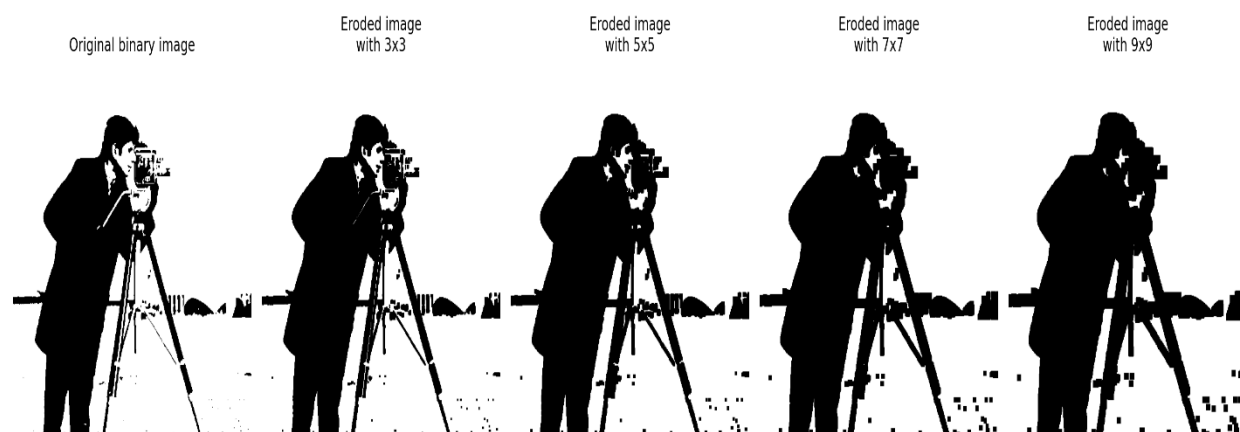
اثر erosion :

- شیء سفید کوچک تر می شود.
- لبه ها عقب می روند.
- بخش های باریک ممکن است حذف شوند.
- اجسام خیلی کوچک ممکن است کاملاً ناپدید شوند.

چرا erosion مفید است؟

- حذف نویزهای سفید کوچک
- جدا کردن اجسام چسبیده
- نازک کردن اشیا

- آماده سازی برای استخراج ویژگی



۳- تمرین سوم: Opening

هدف

حذف اجسام کوچک، حذف برجستگی‌های باریک و صاف کردن شکل اصلی.

ایده‌ی ریاضی

$$A \circ B = (A \ominus B) \oplus B$$

یعنی:

۱. اول erosion

۲. بعد dilation

چرا اول erosion؟ چون erosion اجسام کوچک و قسمت‌های باریک را حذف می‌کند.

چرا بعد dilation؟ چون می‌خواهیم بعد از حذف نویز، اندازه‌ی کلی شیء تا حدی بازسازی شود.

پس opening برای:

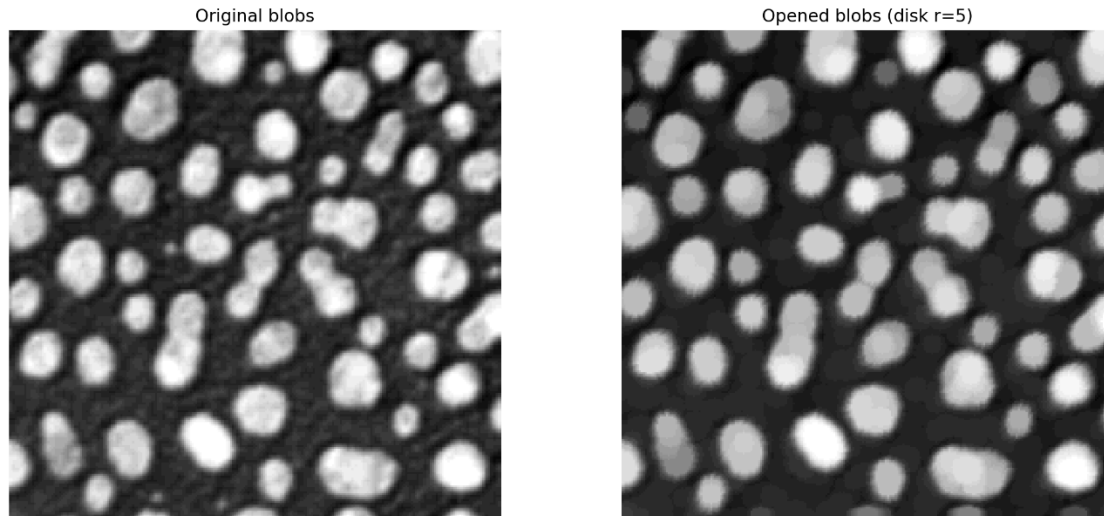
- حذف اشیای کوچک
- حذف noise
- جدا کردن بخش‌های باریک
- نگه داشتن بخش‌های اصلی بسیار مناسب است .

مثال اول : blobs.png

```
I=imread('blobs.png');  
se=strel('disk',5);  
I_opened = imopen(I,se);
```

چرا blobs؟

چون شامل لکه‌ها یا قطعاتی است که بعضی از آن‌ها از بقیه کوچک‌ترند.
opening این لکه‌های کوچک را حذف می‌کند.



چرا disk(5)؟

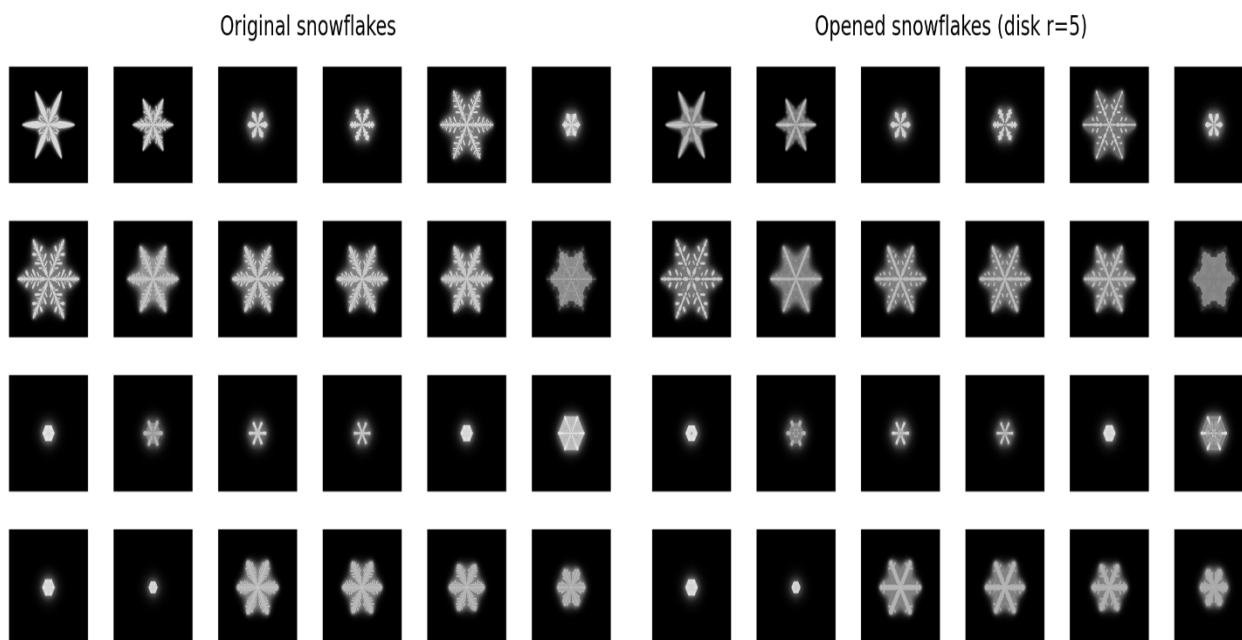
شعاع ۵ یعنی فقط اجزایی که از این مقیاس کوچک‌ترند حذف می‌شوند.
این اندازه تعادل خوبی بین حذف نویز و حفظ ساختار ایجاد می‌کند.

مثال دوم: snowflakes.png

```
I=imread('snowflakes.png');  
se=strel('disk',5);  
I_opened = imopen(I,se);
```

چرا snowflakes؟

چون این تصویر معمولاً شامل اشکال ریز و شاخه‌ای است. Opening اجزای خیلی ریز، زوائد باریک و نویزهای کوچک را حذف می‌کند.



نتیجه:

- اجزای اصلی باقی می‌مانند.
- قسمت‌های نازک یا جداافتاده حذف می‌شوند.
- تصویر تمیزتر و ساده‌تر می‌شود.

۴- تمرین چهارم: Closing

هدف

پر کردن شکاف‌های کوچک، وصل کردن بخش‌های نزدیک و صاف کردن مرزها.

ایده‌ی ریاضی

$$A \cdot B = (A \oplus B) \ominus B$$

یعنی:

۱. اول dilation

۲. بعد erosion

چرا اول dilation و بعد erosion ؟ چون:

- Dilation شکاف‌ها را می‌بندد و بخش‌ها را نزدیک‌تر می‌کند.
- Erosion بعداً شکل را به اندازه‌ی اولیه نزدیک می‌کند اما شکاف‌های بسته‌شده باقی می‌مانند.

پس closing برای:

- بستن gap‌های کوچک
- پر کردن حفره‌های کوچک
- صاف کردن لبه‌ها

کد

(۱) خواندن تصویر circles

```
originalBW = imread('circles.png');
```


چرا circles؟ چون برای closing عالی است: دایره‌ها معمولاً شکاف‌های کوچک یا ناصافی‌هایی دارند که closing آن‌ها را بهتر نشان می‌دهد.

(۲) ساخت SE دیسکی

`se = strel('disk', 10);`

چرا disk؟ چون دایره‌ها شیء اصلی هستند و SE دیسکی از نظر شکلی مناسب‌تر از مربع است.

چرا 10 = radius؟

چون می‌خواهیم gap‌هایی با اندازه‌ی مشخص بسته شوند. اگر شعاع خیلی کوچک باشد، شکاف‌ها بسته نمی‌شوند و اگر خیلی بزرگ باشد، شکل بیش از حد تغییر می‌کند.

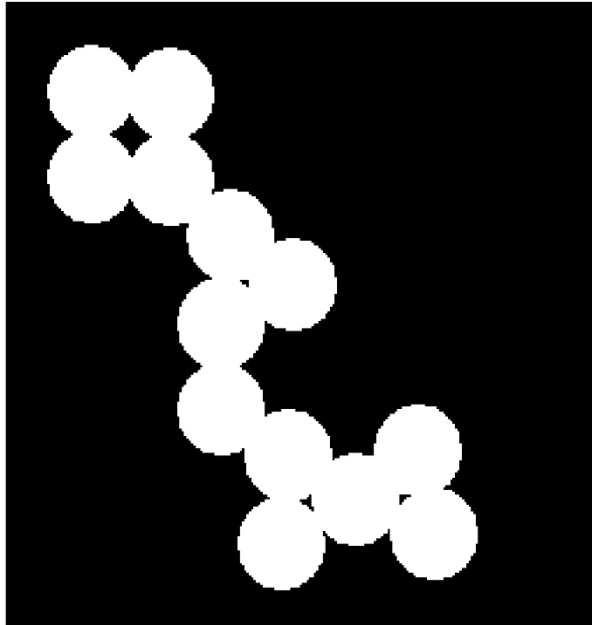
(۳) اعمال closing

`closeBW = imclose(originalBW, se);`

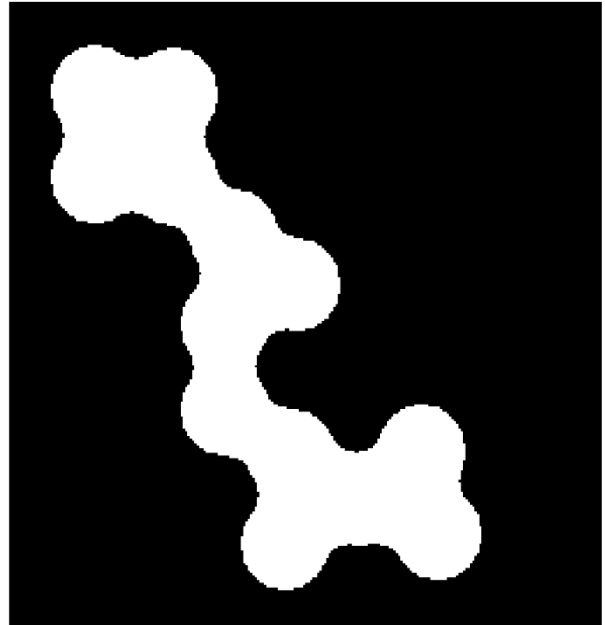
نتیجه:

- فاصله‌های کوچک بین اجزای دایره‌ها پر می‌شود.
- مرزها نرم‌تر و یکنواخت‌تر می‌شوند.
- اجسام به هم نزدیک‌تر و یکپارچه‌تر دیده می‌شوند.

Original circles image



Closed image with disk $r=10$



کاربرد closing چیست؟

- پر کردن حفره‌های کوچک
- بستن شکاف‌ها
- یکپارچه‌سازی قطعات شکسته
- آماده‌سازی برای segmentation

۵- تمرین پنجم: Hit-or-Miss Transform

هدف

تشخیص یک الگوی باینری خاص در تصویر.

ایده‌ی اصلی

Hit-or-Miss برای pattern matching در تصاویر باینری استفاده می‌شود. این عملیات می‌پرسد: "آیا در این نقطه، دقیقاً این شکل خاص وجود دارد یا نه؟"

کد ورودی

```
bw=[000000001100011110011110001100001000];
```

این یک تصویر باینری کوچک مصنوعی است تا بتوان الگو را دقیق بررسی کرد. چرا این ماتریس؟ چون در مثال آموزشی، تصویر کوچک بهتر از تصویر بزرگ است؛ راحت‌تر می‌توان الگو را دید و خروجی را تحلیل کرد.

ماتریس interval یعنی چه؟

```
interval=[0 -1 -1 1 1 -1 0 1 0];
```

در hit-or-miss :

- 1 یعنی این خانه باید حتماً ۱ باشد.
- -1 یعنی این خانه باید حتماً ۰ باشد.
- 0 یعنی فرقی ندارد.

این ماتریس در واقع یک قالب تشخیص الگو است.

چرا این فرمت مهم است؟

چون hit-or-miss فقط وجود pixelهای روشن را نمی‌سنجد، بلکه همزمان وضعیت پس‌زمینه اطراف آن را هم بررسی می‌کند. به همین دلیل برای تشخیص گوشه‌ها، انتهاها، و الگوهای کوچک بسیار مفید است.

خروجی چه چیزی است؟

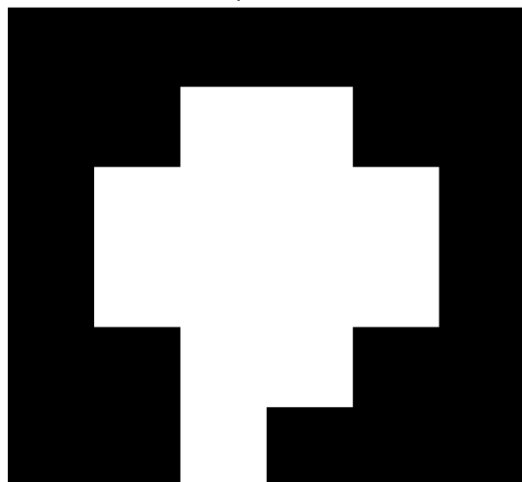
`bw2 = bwhitmiss(bw, interval)`

خروجی، مکان‌هایی را روشن و نشان می‌دهد که الگوی مورد نظر دقیقاً پیدا شده است.

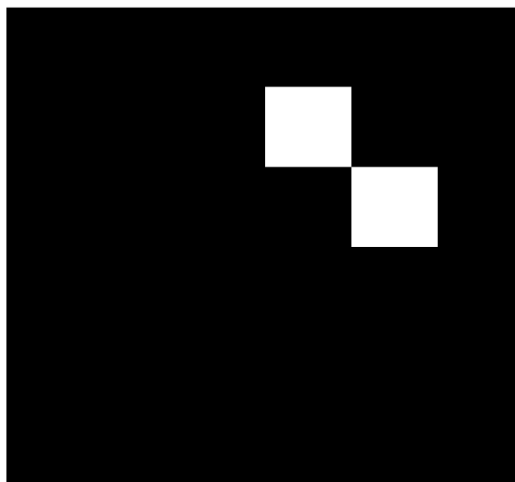
کاربردها:

- تشخیص شکل‌های خاص
- پیدا کردن endpoints در skeletonization
- تشخیص cornerها و اتصال‌ها
- pattern recognition در تصویر باینری

Input bw



bwhitmiss result



جمع‌بندی مفهومی نتایج

Dilation

- اجسام بزرگ‌تر شدند.
- نقاط پراکنده تقویت شدند.
- برای وصل کردن بخش‌های نزدیک مفید است.

Erosion

- اجسام کوچک‌تر شدند.
- بخش‌های باریک حذف شدند.
- برای حذف نویز و جدا کردن اشیا مفید است.

Opening

- اجسام کوچک حذف شدند.
- شکل‌های اصلی حفظ شدند.
- برای تمیز کردن تصویر و حذف نویز عالی است.

Closing

- شکاف‌ها و حفره‌های کوچک بسته شدند و لبه‌ها هموار شدند.
- برای یکپارچه‌سازی اشکال مناسب است.

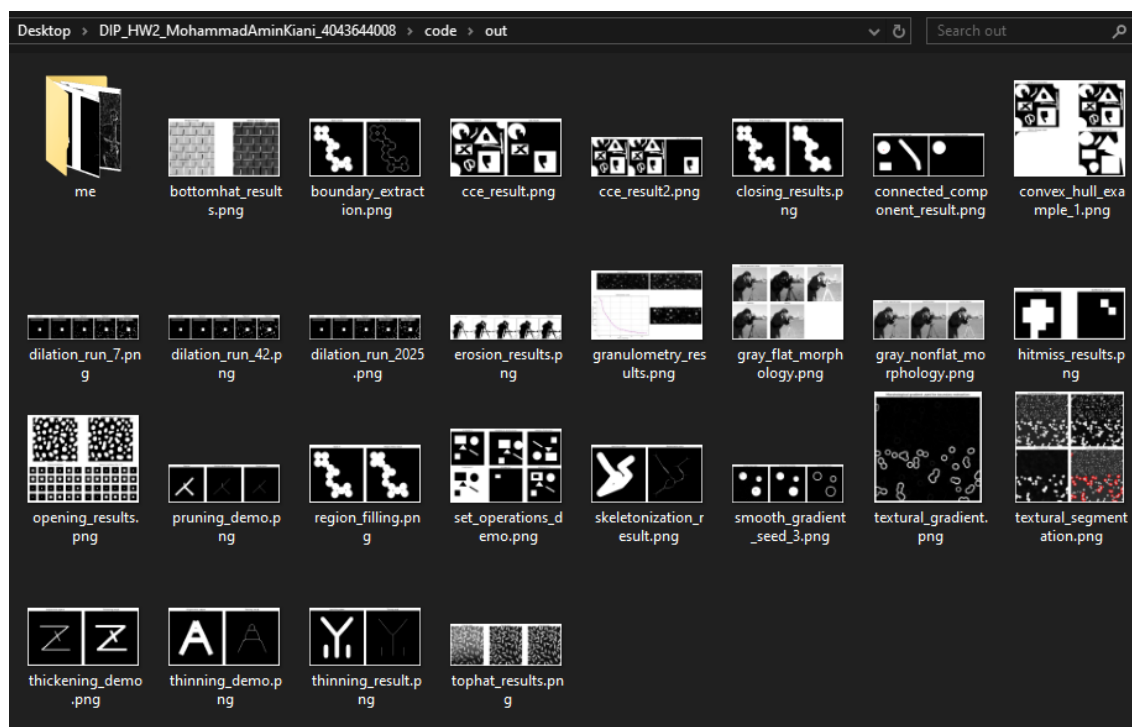
Hit-or-Miss

- الگوی خاص در تصویر شناسایی شد.
- برای تشخیص شکل‌های دقیق و محلی مفید است.

۶- تمرین ششم: باقی عملیات‌های ریخت‌شناسی

هدف، دیدن اثر عملیات پایه مورفولوژیک روی تصاویر دودویی و خاکستری و ترجمه‌ی مستقیم منطق MATLAB به Python بوده است. در این مجموعه تمرین، اثر هر عمل بر ساختار اشیای روشن و تیره دیده شد و نقش structuring element در کنترل رفتار عملیات‌ها به صورت تجربی تحلیل گشت. در بخش‌های دودویی، عملیات‌هایی مانند dilation, convex, region filling, boundary extraction, closing, opening, erosion, hull, connected components, hit-or-miss, thinning, skeletonization, pruning و thickening اجرا شدند. در بخش‌های خاکستری نیز top-hat, bottom-hat, granulometry, flat/nonflat morphology, hat, textural segmentation و پیاده‌سازی شدند. نتایج نشان داد که morphology ابزاری بسیار مناسب برای حذف نویز، بستن شکاف‌ها، تقویت جزئیات محلی، استخراج مرزها، ساده‌سازی شکل و تحلیل اندازه‌ی اجسام است.

نتیجه‌ی کل آزمایش‌ها



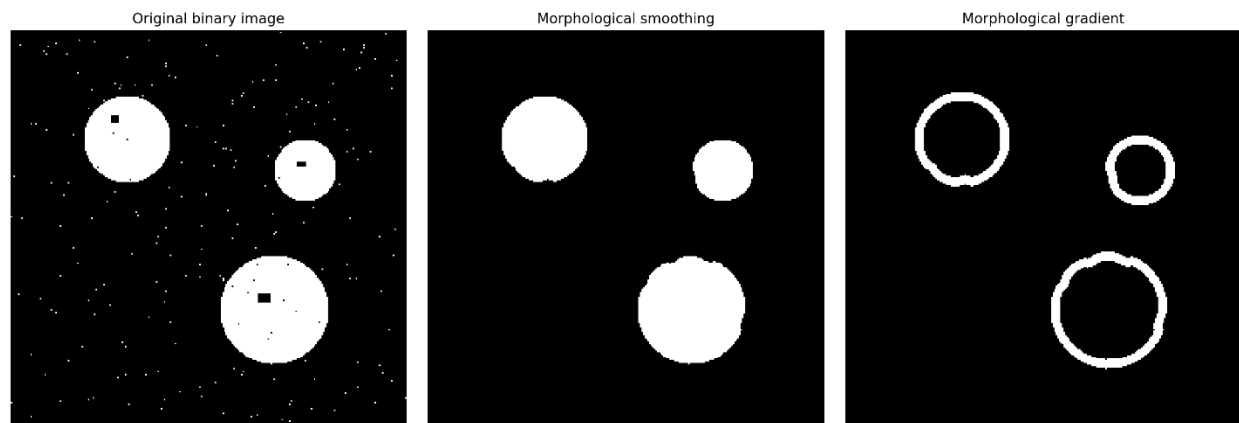
با اجرای تمامی بخش‌ها، این نتیجه‌ها به دست می‌آید:

- Dilation جسم را گسترش می‌دهد و فاصله‌ها را پر می‌کند
- Erosion جسم را کوچک می‌کند و بخش‌های باریک را حذف می‌کند
- Opening نویز کوچک را حذف و شکل اصلی را حفظ می‌کند
- closing شکاف‌ها و حفره‌های کوچک را می‌بندد
- top-hat و bottom-hat جزئیات روشن/تیره‌ی کوچک را برجسته می‌کنند
- boundary extraction مرز جسم را جدا می‌کند
- connected components تعداد و اندازه‌ی اجسام را نشان می‌دهد
- region filling حفره‌ها را پر می‌کند
- convex hull شکل محدب و ساده‌شده‌ی جسم را می‌دهد
- set operations منطق‌های مجموعه‌ای روی تصویر را نشان می‌دهد
- hit-or-miss الگوی دقیق یا endpoint را پیدا می‌کند
- skeletonization، thinning، pruning و thickening ساختارهای خطی و توپولوژی را تحلیل می‌کنند
- granulometry توزیع اندازه‌ی اجسام را آشکار می‌کند
- textural segmentation نواحی بافت‌دار را از هم جدا می‌کند

همچنین شعاع SE باید با مقیاس ساختارهایی که می‌خواهیم حذف یا حفظ شوند هماهنگ باشد:

- disk(3) یا disk(5) برای اثر ملایم
- disk(7) و disk(9) برای اثر متوسط
- disk(10) برای بستن gap های بزرگ تر در circles
- disk(17) برای top-hat / bottom-hat و کار روی جزئیات بزرگ تر روشنایی

Morphological Smoothing and Gradient



smoothed = closing(opening(binary_mask, footprint=se), footprint=se)

این یعنی اول opening و بعد closing .

- **Opening** اجزای کوچک و زوائد باریک را حذف می کند
- **Closing** شکاف های کوچک را می بندد و لبه ها را نرم تر می کند

که این دو با هم یک **smoothing مورفولوژیک** می سازند.

سپس در تصویر بعدی:


```
grad = dilation(smoothed, footprint=disk(3)).astype(np.int16) -  
erosion(smoothed, footprint=disk(3)).astype(np.int16)
```

این همان **morphological gradient** است؛ یعنی اختلاف dilation و erosion .
نتیجه‌اش مرزی است که تغییرات شکل را برجسته می‌کند. این کار برای پیدا کردن لبه و مرز
جسم مفید است.

White Top-Hat و Black Top-Hat

Enhanced grayscale image



Black top-hat / bottom-hat



Enhanced grayscale image



White top-hat

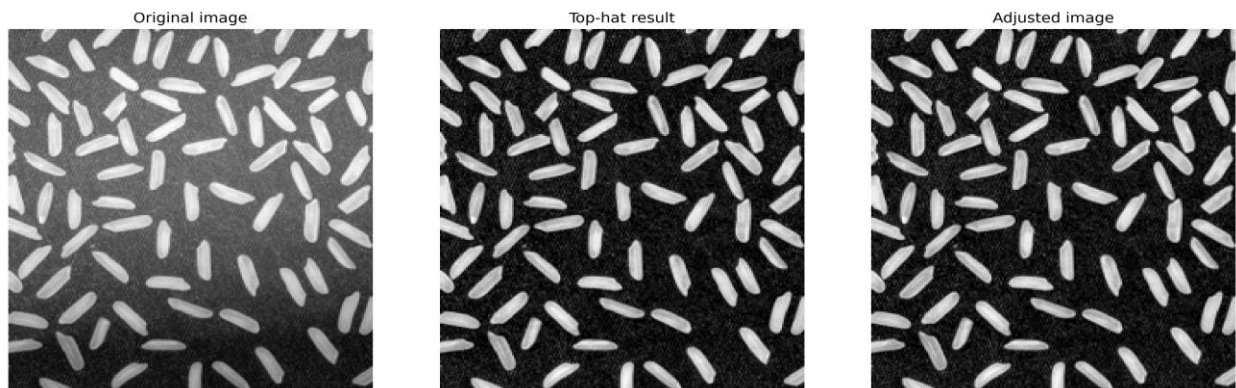


در تابع `op_tophat_bottomhat` از یک `SE` دیسکی با شعاع ۱۷ استفاده شده:

```
se = disk(17)
top = white_tophat(gray_eq, footprint=se)
bottom = black_tophat(gray_eq, footprint=se)
```

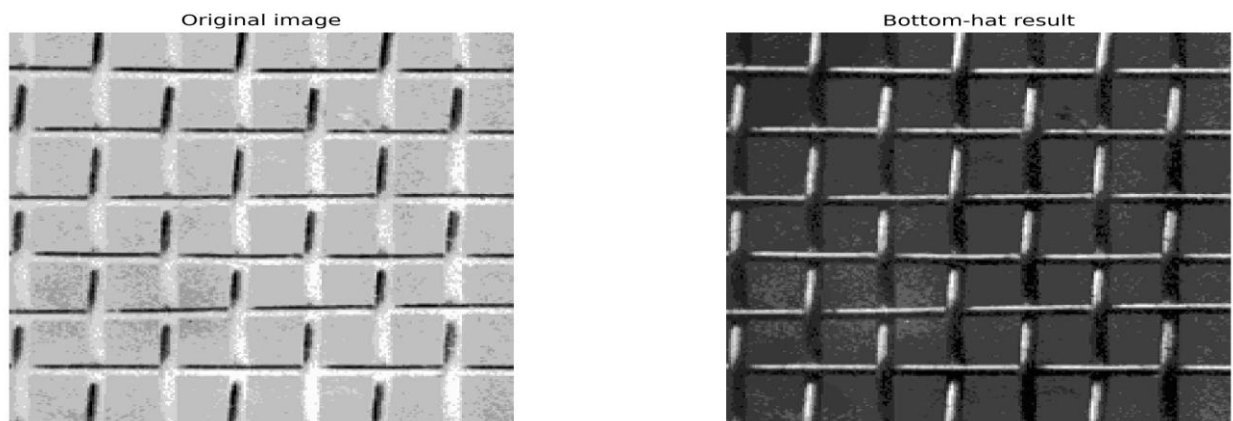
White top-hat

اختلاف بین تصویر و **opening** آن است. یعنی بخش‌های روشنِ کوچک‌تر از **SE** را برجسته می‌کند. کاربردش در تقویت جزئیات روشن، لکه‌های روشن، یا اشیای کوچک روشن است.



Black top-hat

اختلاف بین **closing** و تصویر است. یعنی بخش‌های تیره‌ی کوچک یا فرورفتگی‌ها را برجسته می‌کند. در کد این‌ها روی `gray_eq` اجرا می‌شوند تا هم جزئیات روشن بهتر دیده شوند و هم ناهمگنی روشنایی یا نواحی تیره‌ی کوچک برجسته شوند.



Flat Gray-Scale Morphology

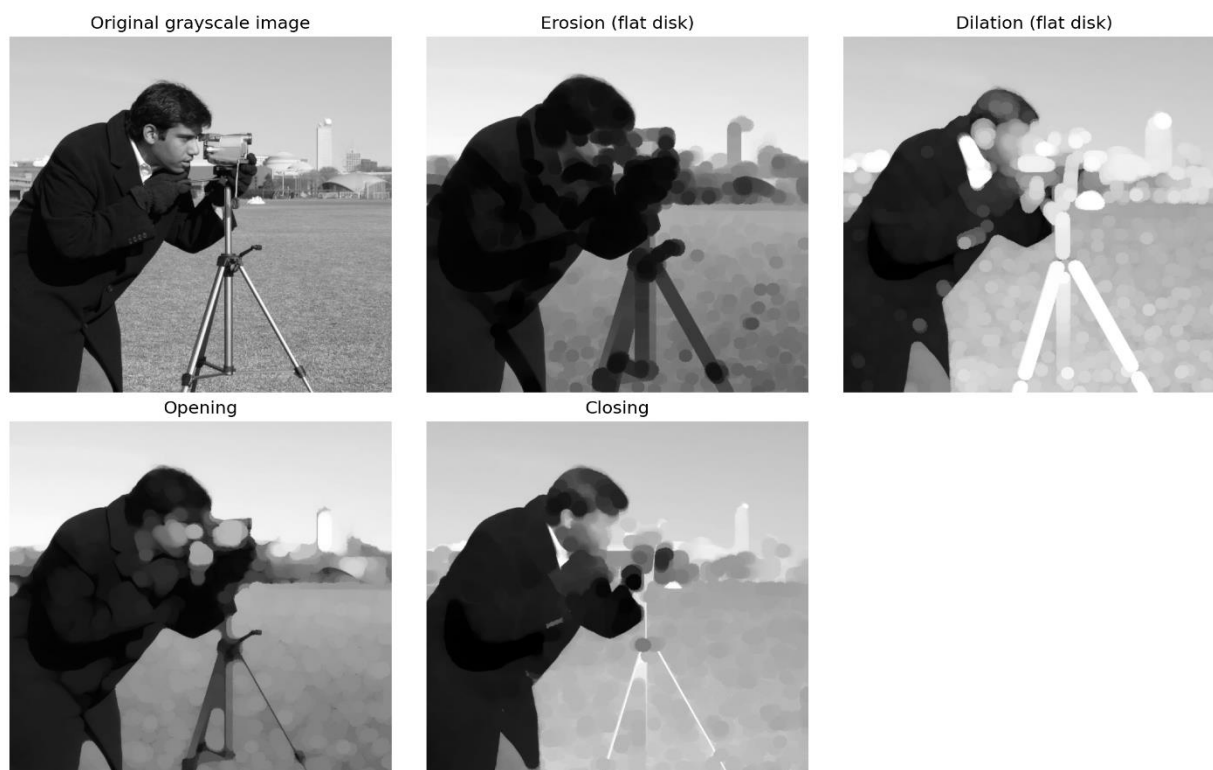
در تابع `op_gray_flat` چهار عمل اصلی روی تصویر `grayscale` اجرا می‌شوند:

<code>ero</code>	<code>=</code>	<code>erosion(gray_eq,</code>	<code>footprint=se)</code>
<code>dil</code>	<code>=</code>	<code>dilation(gray_eq,</code>	<code>footprint=se)</code>
<code>opn</code>	<code>=</code>	<code>opening(gray_eq,</code>	<code>footprint=se)</code>
		<code>cls = closing(gray_eq,</code>	<code>footprint=se)</code>

که در اینجا `se = disk(9)` است.

چرا flat؟

چون `SE` فقط `footprint` دارد و ارتفاع خاصی به آن داده نشده است. در `morphology` خاکستری `flat`، فقط شکل ناحیه‌ی همسایگی مهم است، نه مقدار ارتفاع هر خانه.



Nonflat Gray-Scale Morphology

در تابع `op_gray_nonflat` یک SE غیرمسطح ساخته می‌شود:

```
r = 7
y, x = np.mgrid[-r : r + 1, -r : r + 1]
d2 = x**2 + y**2
structure = -0.02 * d2
footprint = d2 <= r**2
```

اینجا `structure` یک سطح bowl-shaped می‌سازد. یعنی مرکز SE بلندتر و لبه‌ها پایین‌تر هستند. این مدل برای موقعیت‌هایی مفید است که بخواهیم علاوه بر شکل همسایگی، ارتفاع نسبی پیکسل‌ها هم در نظر گرفته شود.

سپس:

```
ero_nf = ndi.grey_erosion(gray_eq, footprint=footprint,
                           structure=structure)
dil_nf = ndi.grey_dilation(gray_eq, footprint=footprint,
                          structure=structure)
```

این‌ها `erosion` و `dilation` خاکستری `nonflat` هستند.

تفاوت مهم این بخش با `flat morphology` این است که شکل SE فقط یک ماسک ساده نیست، بلکه یک سطح واقعی است که روی تصویر اثر می‌گذارد. این برای مدل کردن برخی بافت‌ها و روشنایی‌های غیر یکنواخت مفید است.

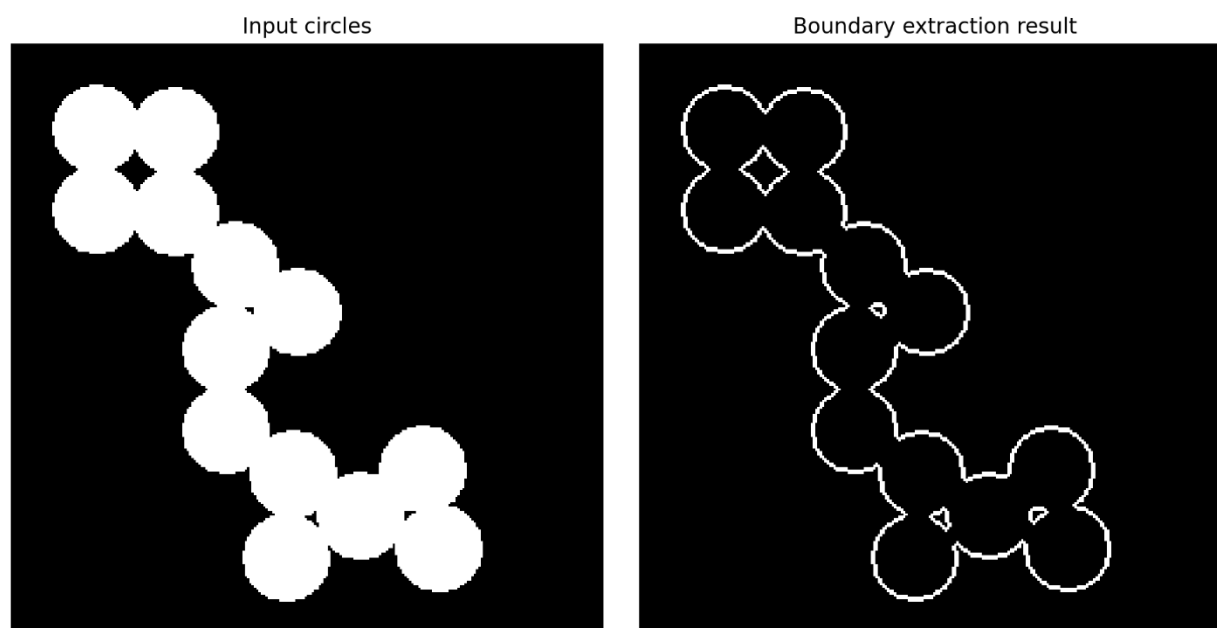


Boundary Extraction

در تابع `op_boundary_extraction`:

```
boundary = binary_mask & (~erosion(binary_mask, footprint=disk(2)))
```

ایده این است که اگر از جسم erosion بگیریم، بخش داخلی کوچکتر می‌شود. حالا با کم کردن erosion از خود جسم، فقط پیکسل‌های مرزی باقی می‌مانند. پس خروجی boundary همان لایه‌ی مرزی جسم است. SE با شعاع ۲ یعنی مرز استخراج شده ضخامت کمی دارد و برای نمایش boundary مناسب است. این مفهوم در استخراج مرز با imsubtract هم آمده بود.



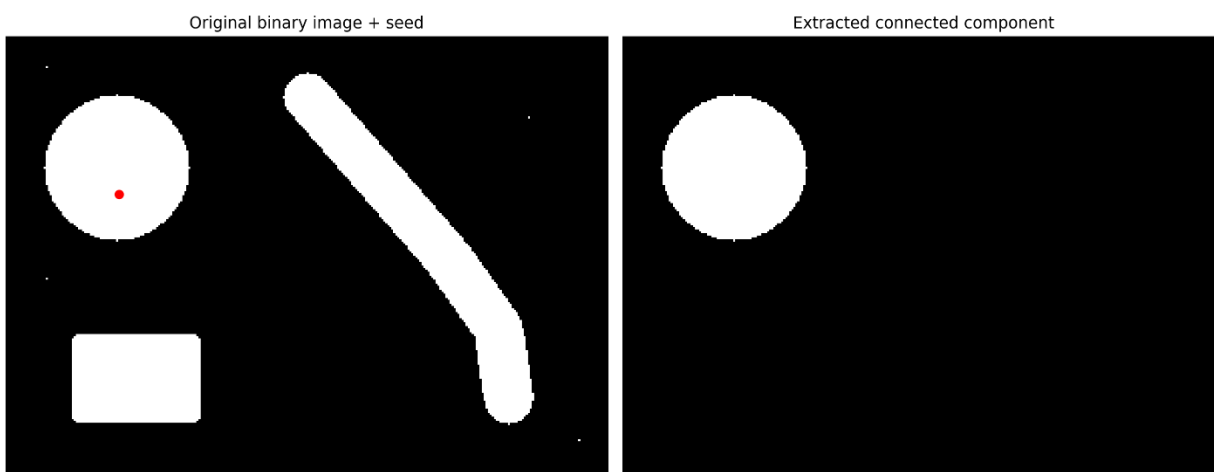
Connected Components

در تابع `op_connected_components` :

```
lab = label(binary_mask)
props = regionprops(lab)
count = len(props)
```

اینجا تصویر باینری به مؤلفه‌های متصل برچسب‌گذاری می‌شود. این کار برای شمارش اجسام جدا، اندازه‌گیری مساحت هر جسم، و پیدا کردن بزرگ‌ترین component مفید است.

سپس تابع `largest_component` فقط بزرگ‌ترین ناحیه را نگه می‌دارد. این کار وقتی مهم است که می‌خواهیم نویزهای کوچک حذف شوند و فقط سوژه‌ی اصلی بماند. در خروجی دوم، `cce_result2.png` هم `bounding box` چند `component` اصلی نمایش داده می‌شود. این برای تحلیل هندسی و `layout` اشیا بسیار مفید است.



Region Filling

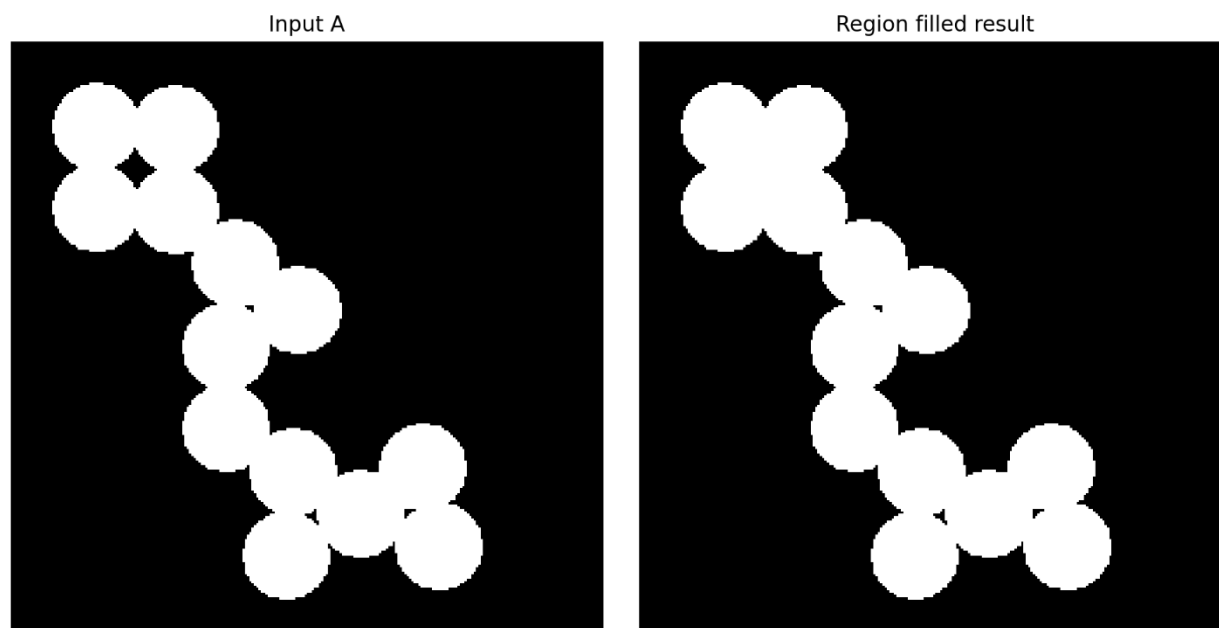
در تابع `op_region_filling`:

```
filled = ndi.binary_fill_holes(binary_mask)
```

این عملیات **حفره‌های داخل اشیا را پر** می‌کند. مثلاً اگر یک دایره‌ی سفید وسطش یک `hole` سیاه داشته باشد، `region filling` آن `hole` را سفید می‌کند. این عملیات برای:

- پر کردن حفره‌ها
- آماده‌سازی `segmentation`
- ساخت `mask` کامل‌تر

خیلی مفید است. در مثال‌های کلاسیک morphology هم region filling یک عملیات پایه‌ای مهم است.



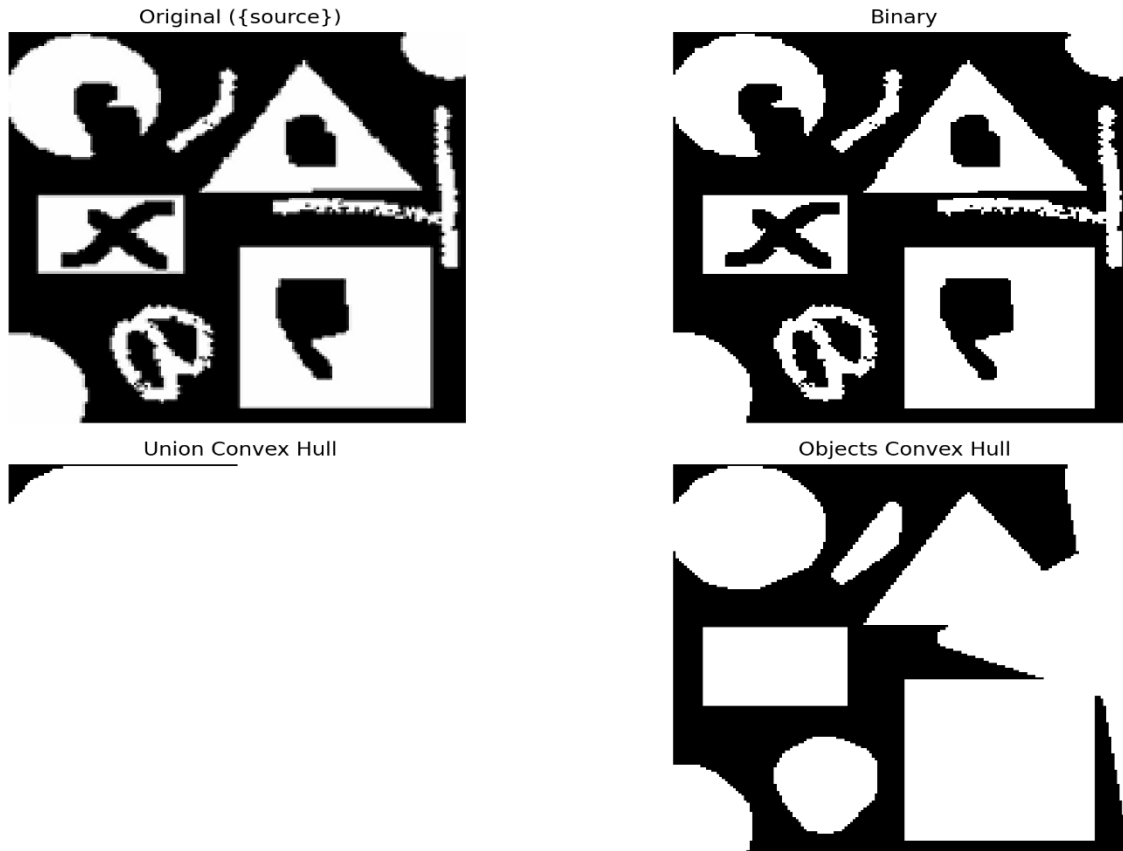
Convex Hull

در تابع `cv2.convexHull` :

```
hull = cv2.convexHull(binary_mask)
```

Convex hull کوچک‌ترین شکل محدب است که جسم را در بر می‌گیرد. یعنی فرورفتگی‌های concave را پر می‌کند. کاربرد:

- ساده‌سازی shape
- استخراج envelope هندسی
- آماده‌سازی برای feature extraction



Set Operations

در تابع `op_set_operations` یک ماسک دوم با شیفت افقی و کمی `dilation` ساخته می‌شود:

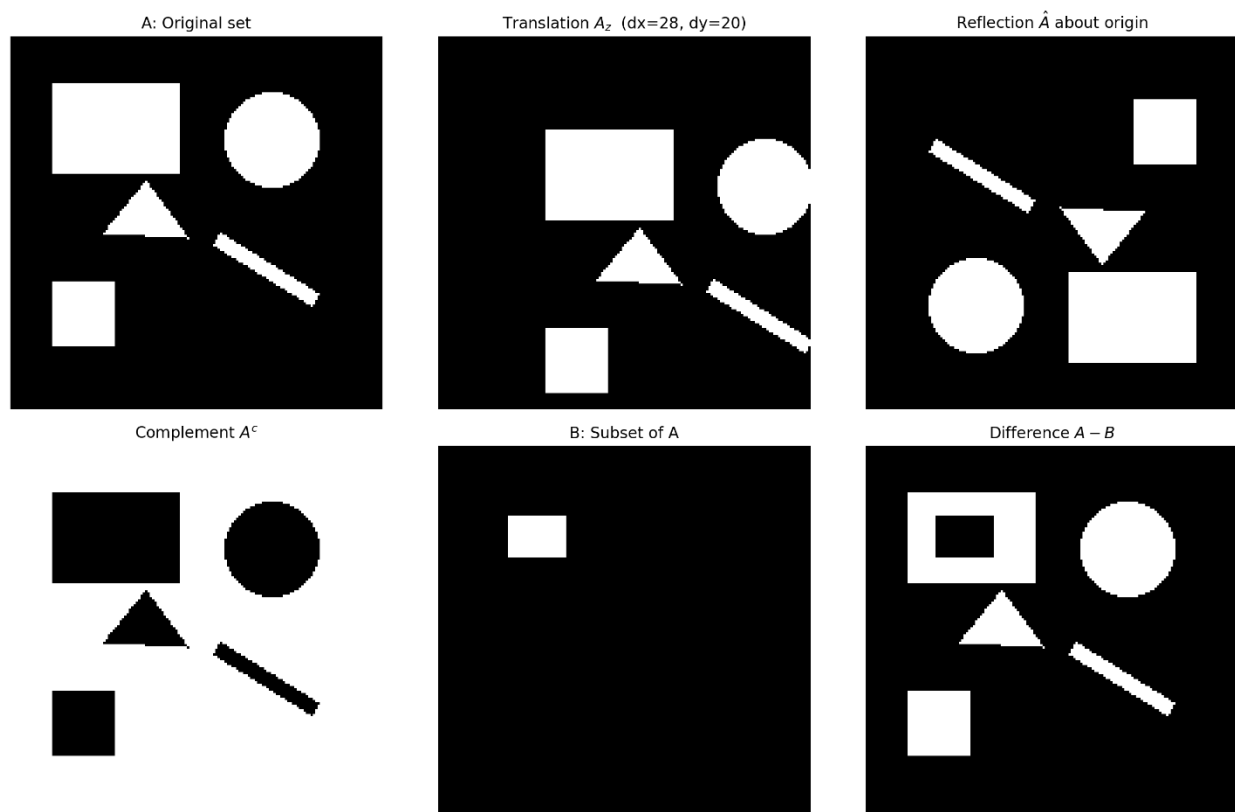
```
shifted = np.roll(binary_mask, shift=22, axis=1)
shifted = ndi.binary_dilation(shifted, structure=np.ones((5, 5),
dtype=bool))
```

این کار عمده دو شیء تا حدی هم‌پوشان می‌سازد تا `union`، `intersection`، `difference` و `XOR` معنی‌دار باشند.

سپس:

- $\text{union} = \text{binary_mask} \mid \text{shifted}$
- $\text{inter} = \text{binary_mask} \& \text{shifted}$
- $\text{diff} = \text{binary_mask} \& (\sim \text{shifted})$
- $\text{xor} = \text{binary_mask} \wedge \text{shifted}$

این بخش برای فهم عملگرهای مجموعه‌ای روی تصاویر باینری است و نشان می‌دهد که تصاویر باینری را می‌توان مثل مجموعه‌های ریاضی تحلیل کرد.



Skeletonization

در تابع `op_skeletonization`:

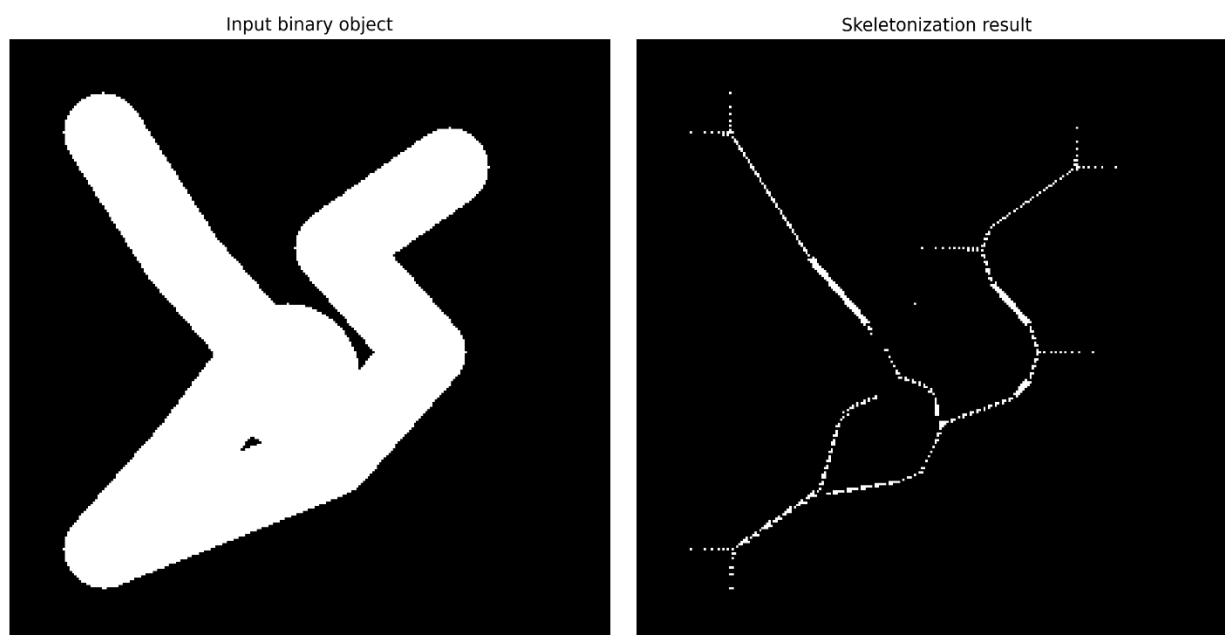
`skel = skeletonize(binary_mask)`

Skeletonization شکل را به یک نمایش میانی medial axis / تبدیل می‌کند. یعنی جسم ضخیم را به خطی باریک‌تر و تقریباً یک پیکسلی تبدیل می‌کند، به طوری که توپولوژی اصلی حفظ شود.

کاربرد:

- OCR
- تحلیل رگ‌ها
- تحلیل مسیرها
- استخراج ساختار اصلی یک شکل

این مرحله برای فهم ساختار کلی اشیا بسیار مهم است.

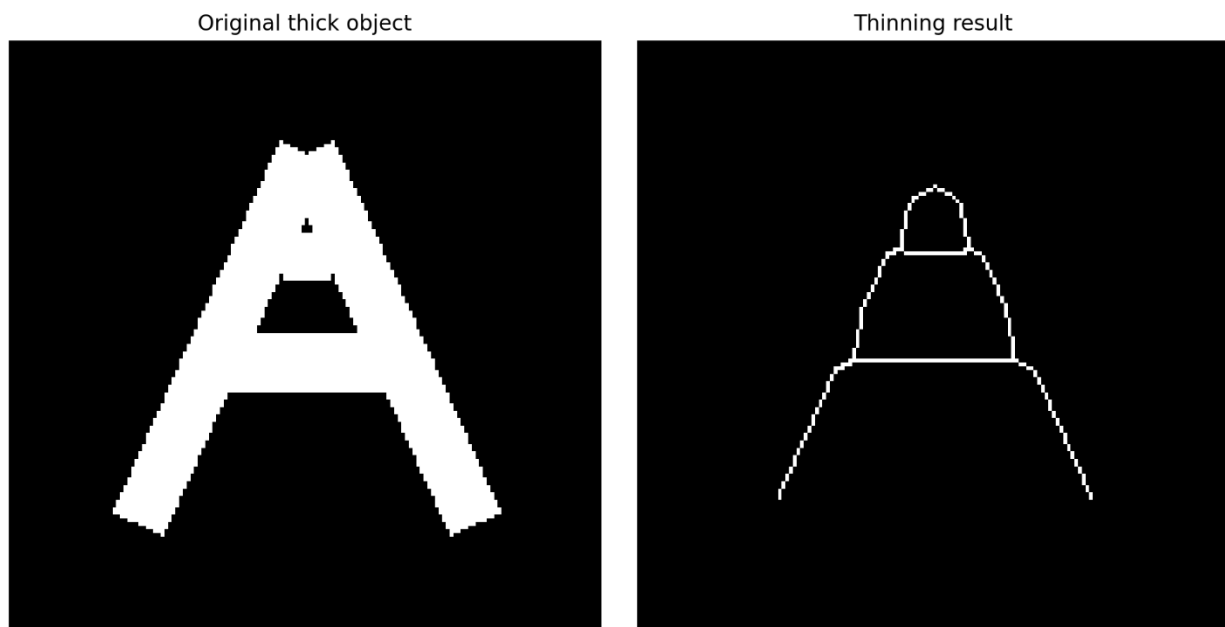


Thinning

در تابع `op_thinning` :

```
thin5 = thin(binary_mask, max_num_iter=5)
thin_final = thin(binary_mask)
```

Thinning در واقع نازک کردن shape است، اما نه لزوماً تا skeleton کامل. در چند iteration پیکسل‌های مرزی به صورت انتخابی حذف می‌شوند تا شکل باریک‌تر شود و connectivity حفظ گردد.



Thickening

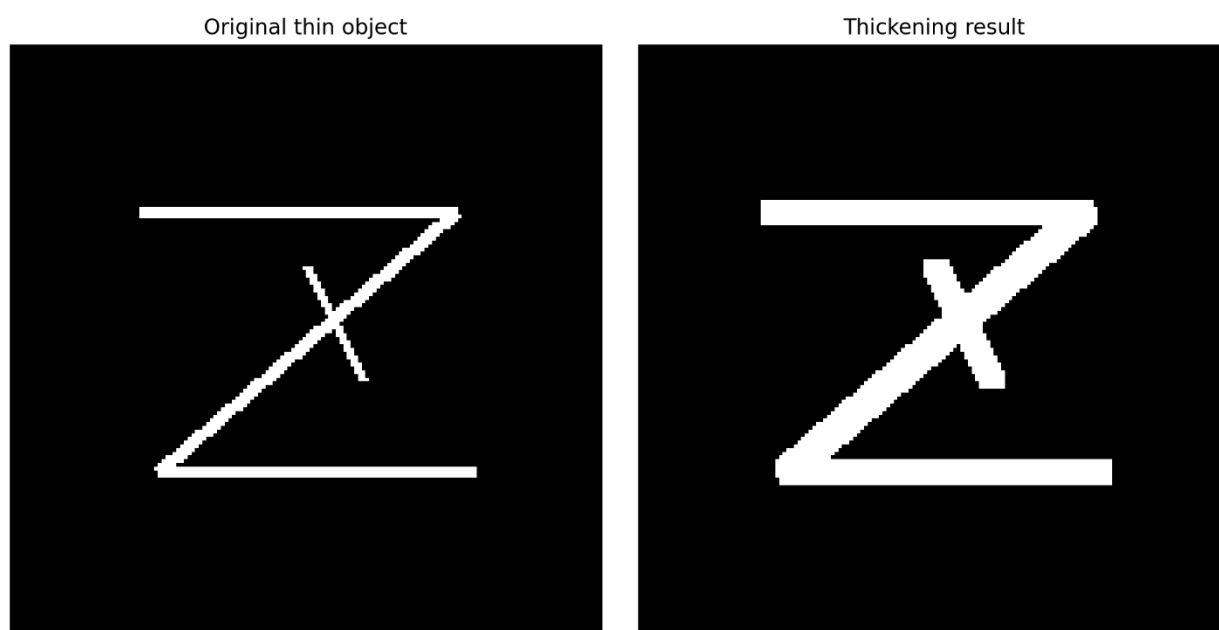
در تابع `op_thickening`:

```
edge = binary_mask & (~erosion(binary_mask, footprint=disk(1)))
thick = edge.copy()
for _ in range(3):
    thick = dilation(thick, footprint=disk(1))
```

اینجا thickening به صورت یک نسخه‌ی عملی و نمایشی ساخته شده است. ابتدا یک لبه یا ساختار نازک به عنوان seed گرفته می‌شود، بعد با dilation چندبار رشد داده می‌شود.

نتیجه:

- ساختار نازک ضخیم‌تر می‌شود.
- برای دیدن اثر رشد مورفولوژیک مفید است.



Pruning

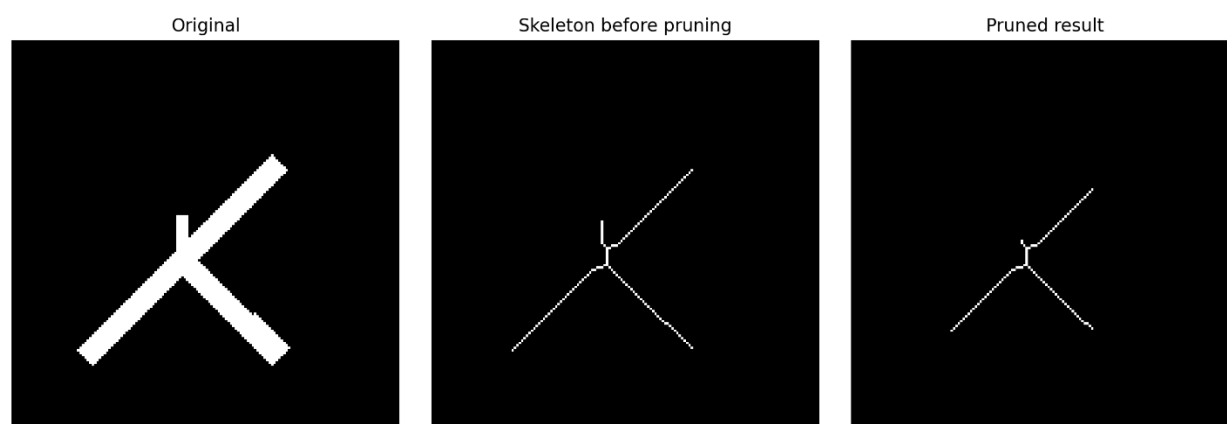
در تابع `op_pruning`:

```
skel = skeletonize(binary_mask)
pruned = prune_skeleton(skel, iterations=12)
```

Pruning یعنی حذف شاخه‌های کوتاه و مزاحم از skeleton. تابع `prune_skeleton` با تشخیص endpointها و حذف تکراری آنها، شاخه‌های نازک و کوتاه را پاک می‌کند.

کاربرد:

- ساده‌سازی skeleton
 - حذف spur های نویزی
 - آماده‌سازی برای تحلیل graph-like structure
- این مرحله معمولاً بعد از skeletonization استفاده می‌شود.



Granulometry

در تابع `op_granulometry` :

```
radii = list(range(1, 22, 2))
for r in radii:
    opn = opening(binary_mask, footprint=disk(r))
    areas.append(opn.sum() / total if total > 0 else 0.0)
```

Granulometry یعنی بررسی توزیع اندازه‌ی اجسام در تصویر. اینجا با `opening` و `SE` های بزرگ‌تر و بزرگ‌تر، اجسام کوچک‌تر حذف می‌شوند و مقدار `area` باقی‌مانده اندازه‌گیری می‌شود.

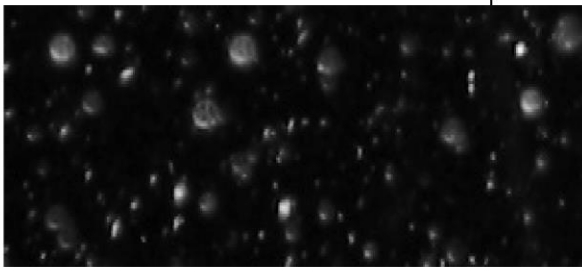
نمودار خروجی نشان می‌دهد که با افزایش شعاع:

- Area کاهش پیدا می‌کند

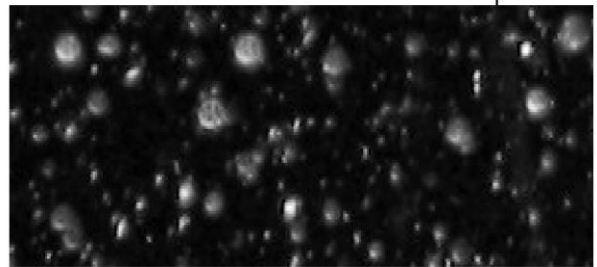
- افت‌های تند، اندازه‌هایی را نشان می‌دهند که در تصویر زیاد حضور دارند

این بخش برای تحلیل اندازه‌ی ذرات، بافت‌ها و **object-size distribution** بسیار مهم است.

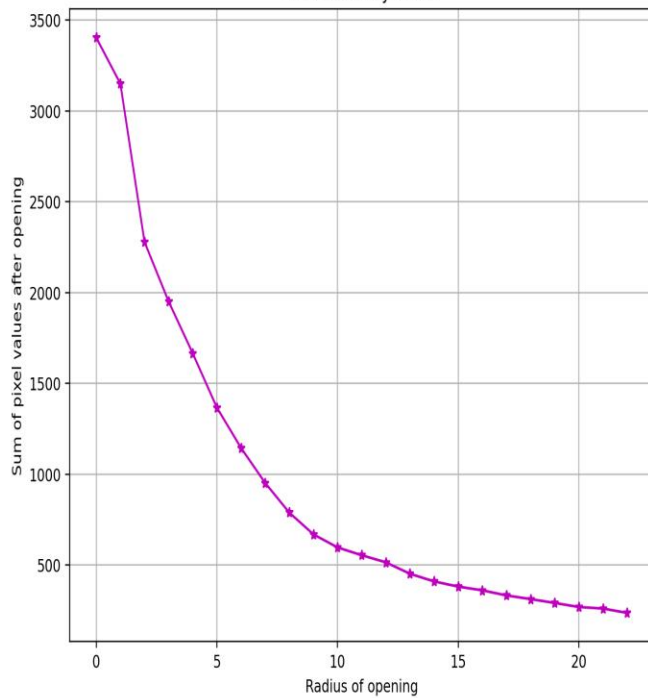
Original image



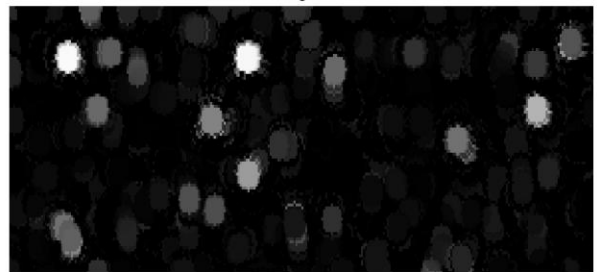
Contrast-enhanced image



Granulometry curve



Size-extracted image ($r=5$ minus $r=6$)



Textural Segmentation

در تابع `op_textural_segmentation` :

```
win = 21
mean = ndi.uniform_filter(gray_eq, size=win)
mean2 = ndi.uniform_filter(gray_eq ** 2, size=win)
local_std = np.sqrt(np.maximum(mean2 - mean ** 2, 0.0))
```

اینجا یک texture map بر اساس انحراف معیار محلی ساخته می‌شود.
اگر یک ناحیه texture بیشتری داشته باشد، انحراف معیار محلی آن بیشتر می‌شود.

سپس:

```
thr = threshold_otsu(tex)
tex_bin = tex > thr
```

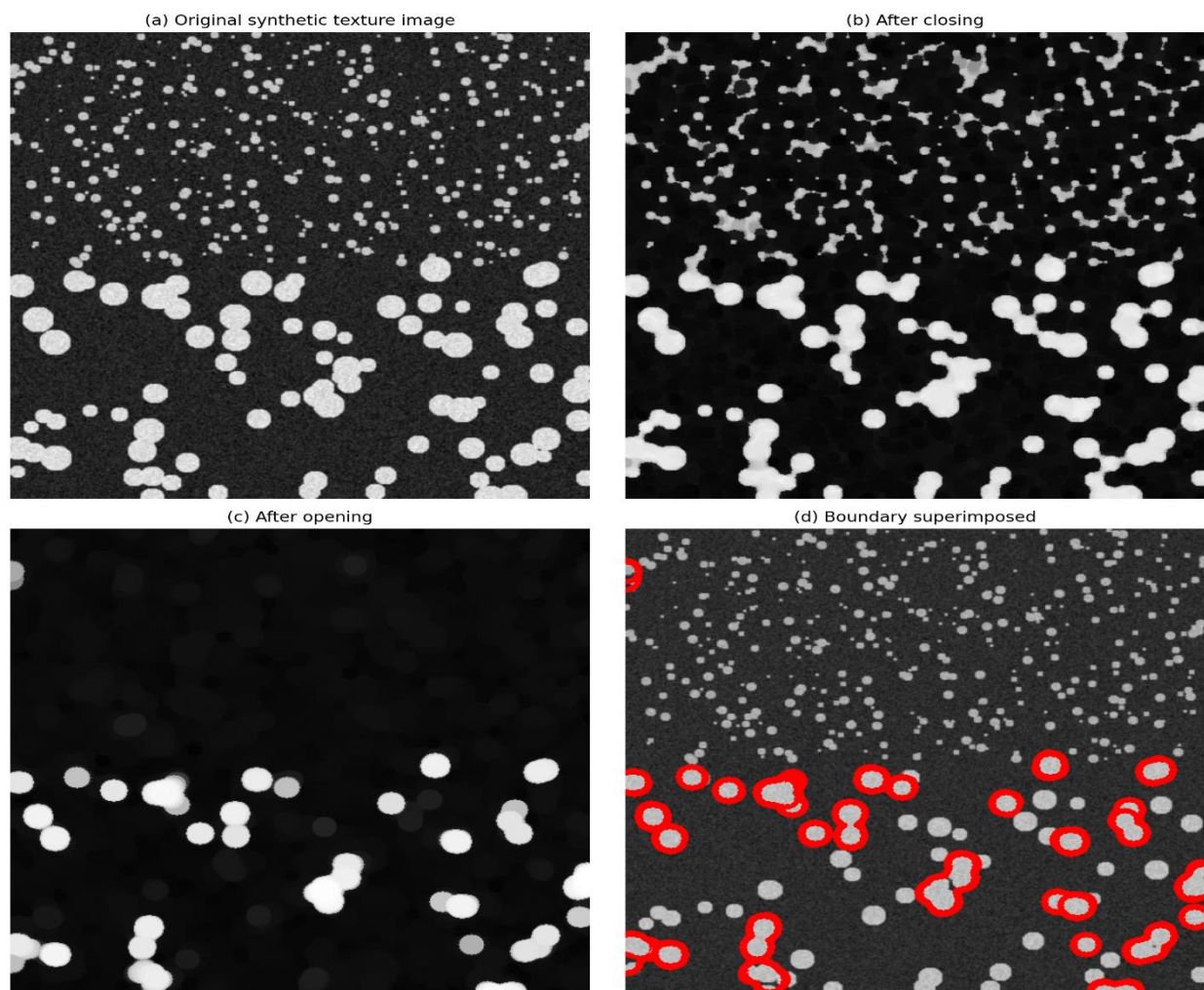
آستانه‌ی Otsu به‌صورت خودکار texture map را به دو کلاس تقسیم می‌کند.

بعد:

```
tex_closed = closing(tex_bin, footprint=disk(5))
tex_opened = opening(tex_closed, footprint=disk(7))
grad = dilation(...) - erosion(...)
```

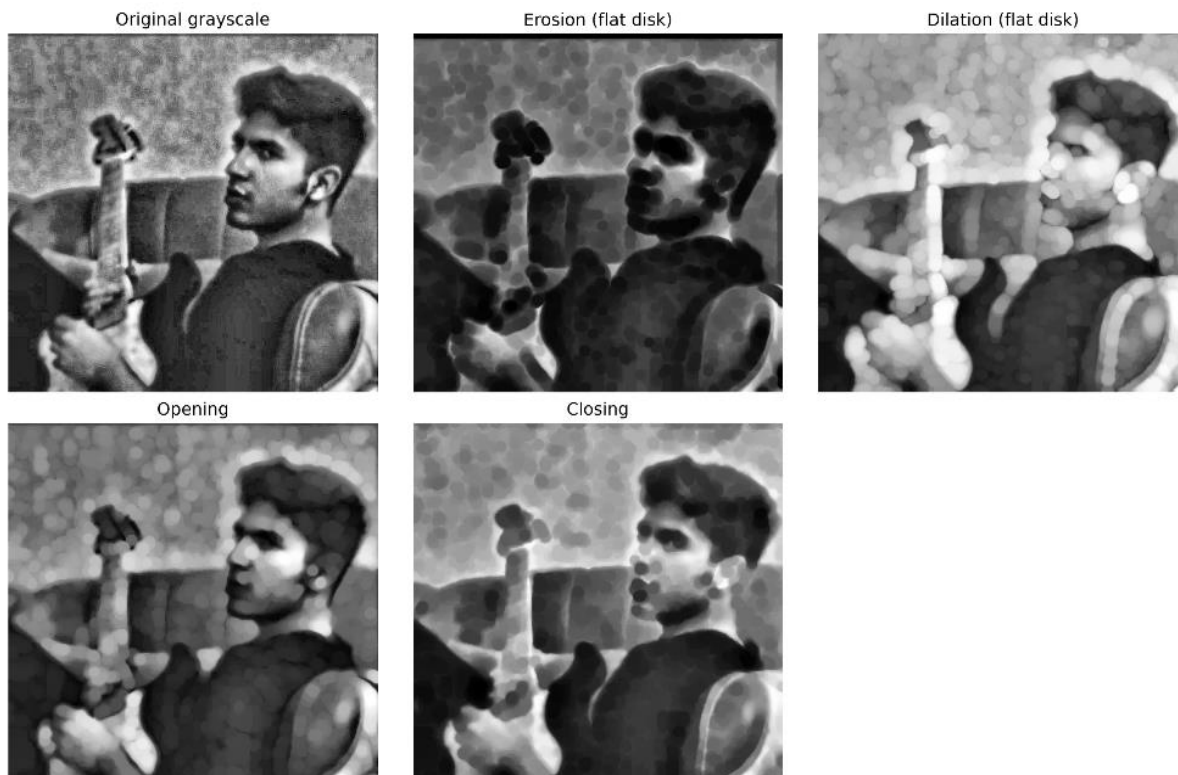
این مراحل برای پاک‌سازی segmentation و استخراج مرزها استفاده می‌شوند.

در نهایت boundary روی grayscale overlay می‌شود تا نتیجه‌ی segmentation دیده شود. این بخش نشان می‌دهد که morphology فقط برای تصاویر باینری نیست، بلکه در segmentation بافت هم نقش مهمی دارد.



۷- تمرین هفتم: انجام تمامی عملیات‌ها بر روی هر تصویر دلخواه

در پیاده‌سازی پایتون، ابتدا یک عکس واقعی خوانده می‌شود، به grayscale تبدیل و با CLAHE تقویت می‌شود، بعد با GrabCut یک mask دودویی از شیء اصلی ساخته می‌شود، و سپس همه‌ی عملیات مورفولوژیک روی همین mask یا روی نسخه‌ی grayscale آن اجرا می‌شوند. این انتخاب باعث می‌شود که هم تعریف نظری عملیات‌ها دیده شود و هم کاربرد عملی آن‌ها روی یک تصویر واقعی مشخص شود.



خواندن تصویر

تابع `load_rgb_and_gray(path)` تصویر را هم به صورت RGB و هم به صورت grayscale می خواند. دلیلش این است که بعضی عملیات ها روی mask دودویی بهتر اجرا می شوند و بعضی دیگر روی تصویر خاکستری.

تبدیل به grayscale و equalization

gray_f نسخه ی نرمال شده ی خاکستری است و `exposure.equalize_adapthist(...)` با CLAHE کنتراست را محلی تقویت می کند. این کار کمک می کند که جزئیات نواحی کم کنتراست در ادامه ی پردازش بهتر دیده شوند.

GrabCut برای ساخت foreground mask

تابع `grabcut_foreground_mask` یک مستطیل اولیه دور سوژه می‌گذارد، بعد با GrabCut پس‌زمینه و پیش‌زمینه را جدا می‌کند. بعد از آن چند عملیات پاک‌سازی انجام می‌شود:

- `binary_closing` برای بستن حفره‌های کوچک
 - `binary_opening` برای حذف نویزهای کوچک
 - `binary_fill_holes` برای پر کردن سوراخ‌ها
 - `clear_border` برای حذف اجزای چسبیده به مرز
 - `label` و `regionprops` برای نگه داشتن بزرگ‌ترین مؤلفه‌ی متصل
- این زنجیره باعث می‌شود ماسک نهایی تمیز و پایدار باشد و عملیات‌های بعدی روی آن معنی‌دار شوند.

توابع کمکی

`largest_component(mask)`

این تابع فقط بزرگ‌ترین `connected component` را نگه می‌دارد. کاربردش زمانی است که ماسک اولیه چند تکه‌ی ناخواسته دارد و ما می‌خواهیم فقط جسم اصلی بماند.

`bbox_of_mask(mask)`

مرز مستطیلی اطراف ناحیه‌ی `foreground` را پیدا می‌کند. این برای تحلیل ابعاد شیء مفید است.

`overlay_boundary_on_gray(gray, boundary)`

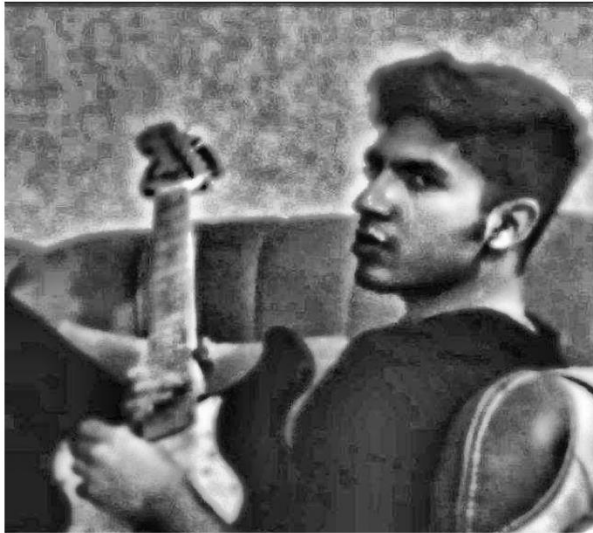
مرز استخراج شده را روی تصویر خاکستری می‌اندازد تا نتیجه بصری تر شود.

`endpoint_mask_from_skeleton(skel)`

این تابع `endpoint` های اسکلت را با شمارش همسایه‌ها تشخیص می‌دهد. ایده‌اش این است که `endpoint` در `skeleton` فقط یک همسایه‌ی `foreground` دارد. این تابع در `hit-or-miss/endpoint detection` و `pruning` استفاده می‌شود.

نکته: تمامی خروجی‌ها در داخل زیر هر سلول از کد و در پوشه‌ی **Out** نیز قرار گرفته‌اند.

(a) Grayscale photo



(b) Texture map (local std)



(c) After closing + opening



(d) Boundary superimposed



- [1] <https://github.com>
- [2] <https://stackoverflow.com/questions>
- [3] <https://colab.research.google.com/>
- [4] <https://www.mathworks.com/help/images/ref/bwconvhull.html>
- [5] Textbook - DIP Gonzalez 4th Edition